

CHAPTER 10

Software Engineering Process

ACRONYMS

BPMN	Business Process Modeling Notation
CASE	Computer-Aided Software Engineering
CMM	Capability Maturity Model
CMMI	Capability Maturity Model Integration
GQM	Goal-Question-Metric
IDEF0	Integration Definition 0
PDCA	Plan-Do-Check-Act
SLCM	Software Life Cycle Model
SLCP	Software Life Cycle Process
UML	Unified Modeling Language

INTRODUCTION

This chapter considers the software engineering process from several perspectives: concepts, life cycles, and software engineering process assessment. The software engineering community has been very active concerning the standardization of many of the aspects of the software engineering process.

BREAKDOWN OF TOPICS FOR THE SOFTWARE ENGINEERING PROCESS

The topic breakdown for the Software Engineering Process KA is shown in Figure 10.1.

1. Software Engineering Process Fundamentals

1.1 Introduction [1,c5],[13]

Software engineering processes involve work activities software engineers conduct to build and operate software. When the discipline of software engineering emerged, scientists, engineers and technicians had to look at existing disciplines to understand the scope of the software engineering process. An engineering process consists of a set of interrelated activities that transform one or more inputs into outputs while consuming resources to accomplish the transformation. As part of engineering, software engineering uses processes similar to those of other types of engineering. As engineers create devices or other products, they progress through various steps, expending significant design effort, relying on a vast trove of knowledge as they do so, at the time that they gain knowledge, i.e. learn, about the process they are performing and the product they are creating.

Beginning in the 1960s and continuing in the 1970s, engineering design and manufacturing provided a baseline — a foundation — for what would later become a new discipline. In those years, it was agreed that the process of building software would be decomposed into processes that could include design and manufacturing, and later, operations. Some of the processes needed to construct software systems fit into the design class, and others fit into the manufacturing class. Today, the software engineering community is still learning and, therefore, still improving the software engineering process. Currently, a wide consensus exists concerning that building software systems requires lots of design and learning effort focused on the product under construction, and on the process. As will be

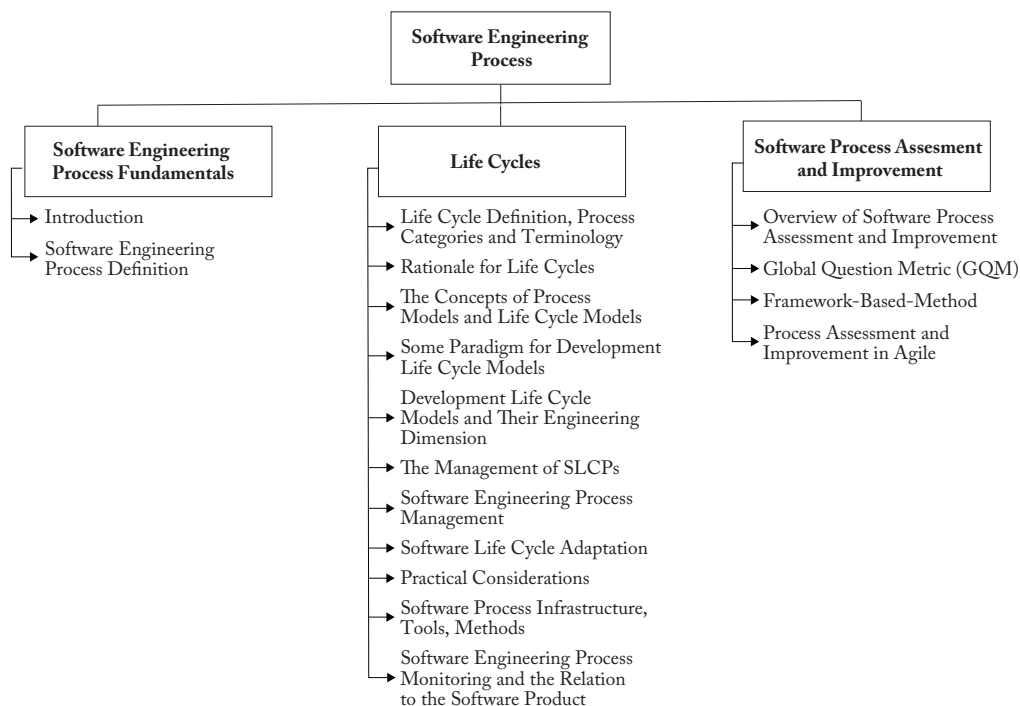


Figure 10.1. Breakdown of Topics for the Software Engineering Process KA

discussed below, no ideal process, or set of processes exists: software processes must be selected, adapted, and applied as appropriate for each project and each organizational context. It is essential that the software engineering process management is supported by empirical measurement.

The concept of a project emerges as an “endeavor with defined start and finish criteria undertaken to create a product or service in accordance with specified resources and requirements” [1] or a “temporary endeavor undertaken to create a unique product, service, or result” [13]. It is a concept of the management discipline linked to clear objectives and bound by a limited time frame, as discussed in The Software Engineering Management KA. Software engineering processes are usually performed in the context of projects.

Many of the processes of the more conventional engineering disciplines (e.g., electrical or chemical) include design and manufacturing, where manufacturing produces multiple units of a system (e.g., a chemical reactor). You do

not produce multiple units in the case for software systems, though manufacturing is useful to describe the need to build the many software units that comprise a software system, and you have a flow of units that can be modeled and managed; likewise, in manufacturing you have a flow of materials. In electrical or chemical engineering, the operation of the engineering systems transforms (raw) materials, energy and physical entities into other forms of material or energy. For the software engineering discipline, an analogy for this operation is the execution of a software unit (the output from a software engineering set of processes) that transforms one kind of data into another.

In the rest of the section, the term *process* will denote work activities, not the execution of software.

The Software Engineering Process KA is closely related to most of the SWEBOK KAs; in particular the Software Engineering Management, Software Engineering Models and Methods, Software Quality, Software Architecture, and Software Testing KAs. The

Measurement and Root Cause Analysis sections in the Engineering Foundations KA are also closely related.

1.2 Software Engineering Process Definition [1,c5] [2] [7][14][20]

A process is a “set of interrelated or interacting activities that transforms inputs into outputs”, where activity is a “set of cohesive tasks of a process,” and a task is a “required, recommended, or permissible action, intended to contribute to the achievement of one or more outcomes of a process” [1]. According to [2], a process is a “predetermined course of events defined by its purpose or by its effect, achieved under given conditions.” A third definition, following [7], is a “system of activities, which use resources to transform inputs into outputs.” And a fourth one is a “set of interrelated or interacting activities which transforms inputs into outputs to deliver an intended result” [20]. That is, the description of a process includes required inputs, transforming activities, and the outputs generated. These definitions address any processes that are applied to the software part of software systems. Software systems also include hardware, and they also involve people and manual procedures. The output of one process can be an input to another process. Processes may include controls (e.g. directives and constraints) and enabling mechanisms (e.g. tools, technologies or resources such as workforce and infrastructure) associated with the processes [14].

2. Life Cycles¹

2.1. Life Cycle Definition, Process Categories, and Terminology [1,c5-6][3*,c2][8*,c1-3][13]

A *life cycle*, according to [1], is the “evolution of a system, product, service, project or other human-made entity from conception through retirement.” In software engineering, life cycles help convey information about software

systems, the “system[s] for which software is of primary importance to the stakeholders” [1]. The concept of life cycles was put in place because simply identifying and defining the processes required to produce software did not adequately describe all the complexity of software systems. It was also necessary to define life cycles, which include a number of processes and constraints [8].

In software engineering, *development* refers to a crucial stage of a system, product, service or project life cycle: that of building (or changing) a software system according to the stakeholders’ needs. From a production/industrial management perspective, software systems are referred to as products. In this context, the term *software product development lifecycle* makes sense.

Product life cycle can be defined as the “series of phases that represent the evolution of a product, from concept through delivery, growth, maturity, to retirement” [13]. This definition is not specific to software systems but applies to all products more generally. Likewise, the life cycle concept, which is linked to the product concept, is not specific to software engineering.

Software systems contain software units that are an “atomic-level software component of the software architecture that can be subjected to stand-alone testing.” (See the Testing KA.) The life cycle of a software system (as long as software engineering uses an interdisciplinary approach), comprises all the processes, activities and tasks from the ideation of the software system to the retirement of the system, including production, operation and evolution, as well as acquisition, when needed, and supply. Likewise, we can look at the life cycle of an element of a software system (a software unit). A software system life cycle will consider both the business and the technical needs of the stakeholders and the system’s ability to produce, as the outcome of the different software life cycle processes (SLCPs) performed by a team, a product that meets the stakeholders’ needs, with the required quality

¹ Lifecycle, life-cycle and life cycle are different spellings. Merriam-Webster prefers the spelling “life cycle”.

level for its users and for all the different stakeholders.

The following paragraphs enumerate the process categories, as in [1]. These process categories reflect the multiple perspectives involved in producing a software system: (1) technical processes including engineering practices to build, make, evolve, operate and retire software products; (2) technical management processes that cover planning and control, as well as configuration management, risk management, information management and quality assurance; (3) organizational project-enabling processes that support life cycle model and infrastructure management, portfolio management, and human resources, knowledge and quality management; and finally (4) agreement processes, which are essential to support collective decision-making, as well as acquisition and supply processes.

A breakdown of these processes is as follows:

1. Technical processes
 - a) Business or mission analysis process
 - b) Stakeholder needs and requirements definition process
 - c) System/software requirements definition process
 - d) Architecture definition process
 - e) Design definition process
 - f) System analysis process
 - g) Implementation process
 - h) Integration process
 - i) Verification process
 - j) Transition process
 - k) Validation process
 - l) Operation process
 - m) Maintenance process
 - n) Disposal process
2. Technical management processes
 - a) Project planning process
 - b) Project assessment and control process
 - c) Decision management process
 - d) Risk management process
 - e) Configuration management process
 - f) Information management process

- g) Measurement process
- h) Quality assurance process

3. Organizational project-enabling processes
 - a) Life cycle model management process
 - b) Infrastructure management process
 - c) Portfolio management process
 - d) Human resource management process
 - e) Quality management process
 - f) Knowledge management process
4. Agreement processes
 - a) Acquisition process
 - b) Supply processes

2.2. *Rationale for Life Cycles* [8*,c2-3][12]

Creating, operating and retiring software products require a number of processes, with their activities and tasks, and a number of constraints. As noted above, software systems involve people and manual procedures, as well as software and hardware. Defining software processes, following [12], requires specifying inputs and outputs. Inputs from processes are, very often, outputs from other processes. Therefore, life cycle processes are interrelated processes; that is, each individual process (its inputs and outputs) may depend on other processes. The interrelated nature of the processes involved make the overall software engineering process highly complex.

The specification of life cycles is a powerful tool for implementing an engineering approach to the creation, operation and retirement of software systems. A life cycle should be defined following engineering principles that guide engineering as a discipline [8]. The specification of a life cycle includes the specification of every process and the associated constraints. The process specification should be useful to humans so that they can communicate with one another using this specification. The specification should be easy to understand and correct because life cycle specifications are the basis for technical and engineering management, including coordination and agreement, measurement, assessment and improvement, and quality management.

2.3. *The Concepts of Process Models and Life Cycle Models* [3*,c2][10*,c2][c2]

Section 2.1 provides a number of software life cycle definitions. In reference [2], a new definition introduces the concept of a *standard* as a commonly accepted guiding document, stating that a “project-specific sequence of activities ... is created by mapping the activities of a standard onto a selected software life cycle model (SLCM).” That is, a life cycle is created in conformance with the life cycle model.

Examples of well-known life cycle models for product development are, among others, the waterfall model, the V-model, the incremental model, the spiral model and the Agile model [2, 3, 10].

2.4. *Some Paradigms for Development Life Cycle Models* [3*,c2-3][8*,c2-3][9*,c1][10*,c1][2][11][12]

Each software system has its own features reflecting the stakeholders’ needs, both business and technical. A suitable life cycle will consider all these needs. As explained in Section 2.3, a software life cycle will be defined in conformance with (partially or fully conforming to) an SLCM. Some authors use the term “development” to refer to SLCM, e.g. “iterative development” instead of “iterative (software) life cycle model”. Types of life cycles are described below.

Predictive life cycles are “a form of project life cycle in which the project scope, time, and cost are determined in the early phase of the life cycle” [13]. Predictive life cycles assume that the set of requirements that will be implemented is a closed set that will not undergo substantive change unless a force majeure occurs.

An *iterative* life cycle is “a project life cycle where the project scope is generally determined early in the project life cycle, but time and cost estimates are routinely modified as the project team understanding of the product increases. Iterations develop the product through a series of repeated cycles,

while increments successively add to the functionality of the product” [3, 8, 13]. The iterations duration is defined for each project. The method chosen (see the Software Engineering Models and Methods KA) would prescribe the role and size of iterations.

In an *evolutionary* life cycle, a product or service changes over its lifetime. It may happen because requirements and customer needs change, but it also may happen because requirements are introduced into the product in successive steps and not as a complete and atomic set [3, 8]. “Successive steps” is a synonym for “iterations.”

An *incremental* life cycle is “an adaptive project life cycle in which the deliverable is produced through a series of iterations that successively add functionality within a predetermined time frame. The deliverable contains the necessary and sufficient capability to be considered complete only after the final iteration” [3, 8, 13]. Incremental life cycles are not always predictive, but they can be. *Incremental development* is a “software development technique in which requirements definition, design, implementation, and testing occur in an overlapping, iterative (rather than sequential) manner, resulting in incremental completion of the overall software product” [2].

Continuous development refers to software engineering practices that allow for frequent releases of new systems (including software) to staging or various test environments through the use of automated tools [8, 9, 11].

A life cycle can enforce a rule that the requirements specifications cannot be changed once the requirements process has been finalized and the customer has agreed to the specifications. This happens, for example, in predictive life cycles. On the other hand, when the life cycle does not preclude changes in the requirements specifications, even after the customer has agreed to them and signed off on them, but allows them to change at any point [upon negotiation of interested parties], then the life cycle is said to be *open to change*. Being open to change is one of the claims of Agile development [9, 10].

2.5. Development Life Cycle Models and Their Engineering Dimension [3*,c2][8*,c2-3][9*,c1][10*,c1][2][11][16][17][18][19][25][26][27]

Several life cycle models have become well known with the development of software engineering since its inception. One model, which became popular early in the history of the discipline, is the waterfall model [3], that falls into the category of *predictive*, described previously. The waterfall model approach for product development uses a number of phases, including requirements, preliminary design, detailed design, coding and testing. It implements a very strict process, in which one phase cannot be started until the previous one is finished. The waterfall model was useful because it introduced a systematization in the development of software systems and, therefore, what could be referred to as an engineering approach to software product development. Many variants or extensions, such as the V-model [3], with many different names and nuances, have been introduced in the history of software engineering. The waterfall model was an early attempt to address the so-called software crisis [3]. The waterfall model is document-driven. Reference [2] defines the waterfall model as the “model of the software development process in which the constituent activities, typically a concept phase, requirements phase, design phase, implementation phase, test phase, and installation and checkout phase, are performed in that order, possibly with overlap but with little or no iteration.” The waterfall model is clearly an example of a predictive life cycle.

Some other paradigms, such as the incremental life cycle, also attempted to address the “software crisis.” In this model (see Section 2.4), different phases occur in an overlapping rather than sequential manner. An incremental life cycle can also be a predictive life cycle. This would mean that the requirements are defined and closed before any other development phase is started. The spiral model, introduced by Boehm, is evolutionary and risk-driven, as opposed to document- or code-driven [3]. Reference [2] defines

the spiral model as a “model of the software development process in which the constituent activities, typically requirements analysis, preliminary and detailed design, coding, integration, and testing, are performed iteratively until the software is complete.” Another popular model is rapid prototyping, a “type of prototyping in which emphasis is placed on developing prototypes early in the development process to permit early feedback and analysis in support of the development process” [2]. The unified process, also known as unified software development process, is an iterative and incremental software development process framework [25]. From the unified process, the rational unified process (RUP®) is documented in [26], and the OpenUP, managed by the Eclipse Foundation [27].

The Agile Manifesto [16] effected a disruption in the software engineering community by creating an abrupt change of mindset. The difference was that Agile Manifesto signatories claimed that the process should be open to change — requirements could be modified at any stage of the development process if users’ needs changed. Communication and mutual trust between team/customer were essential. Signatories claimed that team communication, often face-to-face, and communication with the customer were key. Nevertheless, the Agile Manifesto does not say that documents (e.g. to define requirements) are not needed, documents are needed [9, 10]. Signatories also advocated for small software incremental deliveries, as opposed to projects that applied the waterfall model with a single software delivery at the end of the project after months or years of working. Agile makes a clear distinction between, on the one side, values and principles (e.g., always delivering value to the customer or a commitment to technical excellence) and, on the other, practices (peer programming, sprint planning or retrospective). The Agile mindset [10] is different from the predictive mindset. The Agile mindset is based on a number of values and principles (e.g., the importance of communication, being open to change or commitment to technical excellence and always delivering value to the customer); this focus differentiates

Agile from the predictive mindset, which is more focused on committing to the implementation of the requirements specifications. Agile helps address complexity [8, 10].

Several misconceptions arose around Agile, and some still remain. One is that Agile is a method in itself, which it is not. Another is that Agile is “faster” than waterfall because you need not produce any document. A third one is that Agile has a limited or unstructured set of methods/practices; a chart that enumerates several commonly used Agile methods and practices can be found, for example, in [18]. Several Agile methods became popular, like Extreme Programming for product development, Scrum for project management and others. Even considering the ever rising popularity of the Agile life cycle model to address complex projects, scaling up Agile for large projects and portfolios is still challenging. The perception today is that the Agile Manifesto meant a significant disruption; nevertheless, it is already 20 years old, and some authors think that some of its principles might need to be updated, informed by the experience developers have obtained in the past 20 years [17].

The application of Agile practices has transcended the software engineering process, and the terms *business agility* and *Agile organizations* are now very common [19]. From a software engineering point of view, Agile created an opportunity for the industry to achieve a reengineering and a better alignment of software engineering processes and business strategic processes in organizations. The use of an Agile approach by business processes is a common scenario; this is reflected in the principles of DevOps [11], for example, explained later in this section, and process assessment and improvement in Section 3.

The need to provide more frequent releases, the fact that users’ needs and technological life cycles change more frequently, together with the required alignment of the organizations’ strategic plans with the organizations’ IT operations, has led to the creation of DevOps, defined as a “set of principles and practices which enable better communication and collaboration between relevant stakeholders for the purpose of

specifying, developing, and operating software and systems products and services, and continuous improvements in all aspects of the life cycle” [11]. The ability to provide more releases more frequently, once adequate process management has been defined, has become an advantage that makes companies more competitive.

In the history of software engineering, there has been a lot of controversy over software life cycle models — for example, as seen in debate over the merits of the waterfall model versus the Agile model of software development (See Section 2). This controversy should be understood from a historical perspective; new approaches have been disruptive or considered disruptive, and there has been a lack of empirical measures to support evidence-based discussions about software engineering. This situation has been changing slowly but steadily. Using empirical measures as the basis for making decisions is an essential element of software engineering [4, 8]. See also KA 9, Software Engineering Management, and KA 12, Software Quality.

2.6. The Management of SLCPs [14]

The life cycle for any software system contains a number of stages. According to [14], these stages are the following:

1. Concept: At this stage, stakeholders’ needs are identified, concepts will be explored, and solutions will be proposed.
2. Development: At this stage, requirements representing the users’ needs will be refined, solutions will be created, systems built, and all undergo the needed verification and validation processes.
3. Production: This stage will have a different scope depending on the characteristics of the software system under focus. Generally speaking, it will include the production and testing of the system.
4. Utilization: At this stage, the system operates to satisfy users’ needs.
5. Support: At this stage, developers provide the required actions to achieve a satisfactory operation.

6. Retirement: At this stage, the team follows established procedures to dispose of the system.

The stages are not supposed to be sequential, by any means. Actually, the specification of the life cycle for a software system will include the transitions between these stages. It should be clear that these stages have been identified for a general life cycle. Specific life cycles will have specific stages, meaningful to a particular project's stakeholders; these stages will fit into these general stages.

2.7. *Software Engineering Process Management* [1*,c5][2]

Process management is defined as “direction, control, and coordination of work performed to develop a product or perform a service” [2]. Several management levels govern the software engineering process, as explained in reference [1], see KA 9, Software Engineering Management. The lowest level is the technical processes; the second is the technical management level, which will include project management processes. The third level is the (executive) management level, focused on organizational enabling processes, such as knowledge management, life cycle model management, or portfolio management.

2.8. *Software Life Cycle Adaptation* [5] [14] [23][29]

Each software system has its differential characteristics. These differential characteristics, together with the stakeholders' needs, lead to specific life cycles. This adaptation will include identifying all the relevant characteristics, selecting the appropriate standards or documents internal to an organization, selecting a development strategy/life cycle model, stages, and processes, and documenting the decisions and rationale. The adaptation will not require keeping the names provided in Section 2, or including them all [5, 14, 23]. The ISO/IEC 29110 series, Systems and Software Engineering Standards and Guides for Very

Small Entities (VSEs) [29], is an example of a series derived from ISO/IEC/IEEE 12207.

2.9. *Practical Considerations* [8*,c2-3]

Defining a life cycle process includes the specification of the four categories presented in Section 2. This means addressing technical processes (definition of the processes that will be required), organizational processes (this includes human resources, among other processes), technical management processes (how processes are related, how they are monitored and managed), and agreement processes.

The discipline of software engineering has been evolving since its conception for several reasons. The community has never stopped learning, while the complexity of the products has been ever-increasing. Defining a software life cycle for the development of a product requires considering the characteristics of the product (e.g., stakeholders' needs, product size or complexity) and others external to the product, such as the stakeholders' characteristics. Something that the community has learned is that estimations and measurements are essential. Wrong or uncertain estimations in the context of a life cycle will lead to failure. Accurate estimations are not easy to produce. [8]

A current trend in software engineering is a focus on continuous delivery, supported by realistic process and product estimations and measurements. A helpful lesson engineers have learned is that working with large processes without producing any deliverables along the way increases uncertainty. (See DevOps in Section 2.5.) The Agile mindset has contributed to this and has helped engineers recognize the importance of communication in the process. [8]

When a project process is defined in conformity with a life cycle, it is important to make sure that it will be possible to have metrics/measure definitions that will result in realistic process (and product) estimations and measurements throughout project definition and execution, and to define the level of precision and uncertainty; project process (and product) measurements should always provide

accurate information about what is happening (the status of the process and the product) while the life cycle process is executed. If the accuracy of estimations and measurements is uncertain, the project might not be successful. In this case, a reflection should take place on the overall approach. Historically, a lot of polemics have grown about the predictive life cycle versus the Agile life cycle. In software engineering, discussions should always be supported by realistic process and product estimations and measurements, which can accurately reduce the level of uncertainty.

2.10. *Software Process Infrastructure, Tools, Methods* [3*,c2][8*,c2-3][2]

Several notations have been used for defining software processes, including natural language, specifying textual lists of constituent activities and tasks, data-flow diagrams, state charts, integration definition (IDEF0) [30], Petri nets, unified modeling language (UML) activity diagrams, and business process model and notation (BPMN) [2, 3]. Software process infrastructure includes tools to support the definition of these processes (e.g., a BPMN toolkit) but mainly to support all specific processes (testing or configuration management). Process definitions will often include methods and formalism (e.g., Rational Unified Process or extreme programming) [3]. Tools will, ideally, have to support these methods and, as important, be integrated with them. Therefore, it is not enough that a tool supports testing. Once a code unit has been successfully tested, for example, this becomes useful information that should be recorded so that the rest of the team can be aware of this fact. This means that the configuration management tool and the testing tool will have to be integrated [3, 8]. The term *software engineering environment*, representing a set of integrated tools, is sometimes used. The term *CASE* (computer-aided software engineering) was popular in the 1980s and 1990s. Somehow, the power tools of the 1980s and 1990s were oversold as a cure for the software crisis. In any case, today, the automation of some processes (e.g., configuration management, or at least version control;

testing; ticket management) is seen as essential for the implementation of successful life cycles. See also the KA 11, Software Engineering Models and Methods.

2.11. *Software Engineering Process Monitoring and its Relationship with the Software Product* [1,c5-6][3*,c2][4*c4-10][8*c2-3]

Developers must monitor the software engineering process execution, assess whether the process objectives are met, and assess risks. This process monitoring is part of software engineering process assessment (see Section 3) and part of the Software Engineering Management KA [1, 3, 4, 8].

Empirical methods support process assessment and improvement as well as product assessment and improvement. The goal of process execution is to obtain products that meet stakeholders' needs. While this area is focused on the software engineering process, process monitoring requires assessing both process and product, using a joint, more holistic approach.

3. **Software Process Assessment and Improvement**

3.1. *Overview of Software Process Assessment and Improvement* [4*,c4][15][24]

The idea that any executed process can be improved was present in the classic Shewhart-Deming plan-do-check-act (PDCA) paradigm [15, 24], which was already being discussed and applied in the 1950s, and its foundations can be found centuries earlier. For the software engineering process, several approaches have been developed.

The PDCA paradigm is an opportunity to meet a widely recognized need — the need for empirical evidence to make decisions. Such decisions include choosing a life cycle, deciding how to assess a process or deciding how to improve a process, among others. Getting empirical evidence across the execution of a software engineering process is essential for the success of the process execution. [4]

3.2. Goal Question Metric (GQM) [21]

The GQM approach [21] is based on Basili’s Quality Improvement Paradigm. Both are based on setting goals that can be measured, changing something, and then evaluating the effect of the change. When the evaluation is positive, an improvement has occurred.

3.3 Framework-Based Methods [4*,c4-10][6][22]

Some assessment methods are based on frameworks that use a process reference model and an assessment reference model — for example, CMM® (capability maturity model), CMMI® [4, 22], and the ISO/IEC 33000 [4, 6] series, also known as SPICE.

The ISO/IEC 33000 framework includes a process reference framework and a process assessment model. The ISO/IEC 33000 framework revises the ISO/IEC 15504 series of International Standards, providing a framework for the assessment of (1) the process quality characteristics, one of which is process capability, together with (2) organizational maturity. This framework covers processes for the development, maintenance and use of systems across the information technology domain, as well as processes for the design, transition, delivery and improvement of services. The concept of seeking continuous improvement underlies the assessment.

This series has developed several groups of standards addressing, as well as core elements,

basic requirements for performing process assessments, process models and the process measurement framework; guidance on how to perform assessments; measurement frameworks for the assessment of process capability and organizational maturity; process reference models for special cases such as safety or high maturity; process assessment models for SLCPs, system life cycle process IT service management, safety and high maturity; and organizational maturity models.

The process reference model is defined as a “model comprising definitions of processes in a domain of application described in terms of process purpose and outcomes, together with an architecture describing the relationships between the processes.” The process assessment model is defined as a “model suitable for the purpose of assessing a specified process quality characteristic, based on one or more process reference models.” [6]

3.4. Process Assessment and Improvement in Agile [9*,c11][28]

Agile methods (e.g., the scrum project management method) introduce what they call *retrospectives* at the end of each iteration. The objective of the retrospective is to analyze what went well and what did not go well, to understand why, and to set a number of actions for learning and improvement. In the end, the team is in a continuous learning loop [9]. This practice, with different names and scopes was not new in software engineering [28].

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	ISO/IEC/ IEEE 12207 [1]	Sommerville [3*]	Laporte et al. [4*]	Farley [8*]	Shore et al [9*]	PMI [10*]	Others
1. Software Engineering Process Fundamentals							
1.1 Introduction	c5						[13]
1.2 Software Engineering Process Definition	c5						[2] [7][14][20]

2. Life Cycles							
<i>2.1 Life cycle definition, process categories and terminology</i>	c5-6	c2		c1-3			[13]
<i>2.2 Rationale for life cycles</i>				c2-3			[12]
<i>2.3 The concept of process models and life cycles models</i>		c2				c2	[2]
<i>2.4 Some paradigms for development life cycle models</i>		c2-3		c2-3	c1	c1	[2] [11] [13]
<i>2.5 Development life cycle models and their engineering dimension</i>		c2		c2-3	c1	c1	[2] [11] [16] [17] [18] [19] [25] [26] [27]
<i>2.6 The management of SLCPs</i>							[14]
<i>2.7 Software engineering process management</i>	c5						[2]
<i>2.8 Software life cycle adaptation</i>							[5] [14] [23] [29]
<i>2.9 Practical considerations</i>							
<i>2.10 Software process infrastructure, tools, methods</i>		c2		c2-3			[2]
<i>2.11 Software engineering process monitoring</i>	c5-6	c2	c4-10	c2-3			
3. Software Process Assessment and Improvement			c4-10				
<i>3.1 Overview of software process assessment and improvement</i>			c4				[15] [24]
<i>3.2 Goal-question metric (GQM)</i>							[21]
<i>3.3 Framework-based methods</i>			c4-10				[6] [22]
<i>3.4 Process Assessment and improvement in Agile</i>					c11		[28]

REFERENCES

- [1] ISO/IEC/IEEE 12207:2017 Systems and software engineering — Software life cycle processes.
- [2] “ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary,” Edition 2, 2017.
- [3] I. Sommerville, *Software Engineering*. 10th ed. 2016.
- [4] C. Y. Laporte and A. April, *Software Quality Assurance*, IEEE Computer Society Press, 1st ed., 2018.
- [5] Project Management Institute, *Software Extension to the PMBOK® Guide — Fifth Edition*, 2013.
- [6] ISO/IEC 33001:2015 Information technology — Process assessment — Concepts and terminology.
- [7] ISO/IEC 25000:2014 Systems and software engineering — Systems and software product quality requirements and evaluation (SQuARE) — Guide to SQuARE.
- [8] D. Farley, *Modern Software Engineering: Doing What Works to Build Better*

- Software Faster*. Addison-Wesley Professional, 2021.
- [9] J. Shore and S. Warden, *The Art of Agile Development*, O'Reilly Media, 2nd ed. 2021.
- [10] Project Management Institute, *Agile Practice Guide*. Project Management Institute and Agile Alliance. 2017.
- [11] ISO/IEC/IEEE 32675:2022 Information technology — DevOps — Building reliable and secure systems including application build, package and deployment.
- [12] ISO/IEC/IEEE 24774:2021 Systems and software engineering — Life cycle management — Specification for process description.
- [13] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide) — Sixth Edition*.
- [14] ISO/IEC/IEEE 24748-1:2018(E) Systems and software engineering — Life cycle management — Part 1: Guidelines for life cycle management.
- [15] W.A. Shewhart and W.E. Deming, *Statistical Method from the Viewpoint of Quality Control*. Dover, New York, 1986.
- [16] “The Agile Manifesto.” <https://agilemanifesto.org>. [Accessed March 5, 2022].
- [17] S. McConnell, *More Effective Agile: A Roadmap for Software Leaders*, 2019.
- [18] “Subway Map to Agile Practices.” Agile Alliance. <https://www.agilealliance.org/agile101/subway-map-to-agile-practices/> [Accessed March 5, 2022].
- [19] J. Eckstein and J. Buck, *Company-wide Agility with Beyond Budgeting, Open Space & Sociocracy: Survive & Thrive on Disruption*, 2021.
- [20] ISO/IEC TR 29110-5-3:2018 Systems and software engineering — Lifecycle profiles for very small entities (VSEs) — Part 5-3: Service delivery guidelines.
- [21] N. Fenton and J. Bieman, *Software Metrics*, 3rd ed. CRC Press, 2014.
- [22] CMMI Institute — CMMI V2.0. <https://cmmiinstitute.com/cmmi>. [Accessed 5 March 2022].
- [23] ISO/IEC/IEEE 24748-3:2020. Part 3: Guidelines for the application of ISO/IEC/IEEE 12207 (software life cycle processes).
- [24] D.R. Kiran, *Total Quality Management*. Elsevier, 2017.
- [25] J. Rumbaugh, G. Booch, I. Jacobson. *The Unified Software Development Process*, 1999
- [26] P. Kruchten. *The Rational Unified Process: An Introduction*. 3rd Ed. 2004.
- [27] The Eclipse Foundation <https://www.eclipse.org/org/foundation/> [Accessed 25 April 2024].
- [28] T. Dingsøyr, Postmortem reviews: purpose and approaches in software engineering, *Information and Software Technology*, vol. 47, Issue 5, 2005, Pages 293-303.
- [29] ISO/IEC 29110-1-1:2024 Systems and software engineering - Lifecycle profiles for very small entities (VSEs) Part 1-1: Overview.
- [30] ISO/IEC/IEEE 31320-1:2012 Information technology — Modeling Languages Part 1: Syntax and Semantics for IDEF0